INTEGRATION, the VLSI journal 99 (2024) 102244

Contents lists available at ScienceDirect

Integration, the VLSI Journal

journal homepage: www.elsevier.com/locate/vlsi

An area efficient 64 point Radix-4^2 MDC FFT architecture for OFDM applications

M. Srinivasa Rao [a,*], G.L. Madhumati [b], M. Sailaja [a]

[a] University College of Engineering, Jntuk, Kakinada, India
[b] Velagapudi Ramakrishna Siddhartha Engineering College Deemed to be University, Vijayawada, India

ARTICLE INFO

Keywords:
MDC
Radix-4^2
Constant multiplier

ABSTRACT

In this research,we present a 64-point radix-4^2 pipelined Fast Fourier Transform(FFT) architecture which is area-efficient for an orthogonal frequency division multiplexing(OFDM) based IEEE 802.11a wireless Local area network(LAN) baseband. We adopt Multiple Delay Commutator(MDC) architecture for hardware implementation. The proposed 64-point FFT adopts a modified constant multiplier to compute complex multiplication in place of complex multipliers and to avoid read-only memory(ROM),which is used to store twiddle factor coefficients internally. The area of the design is reduced by using modified constant multiplier. The proposed radix-4^2 pipelined FFT architecture is synthesized using 45 nm CMOS technology with a supply voltage of 1.1 V. The proposed design occupies 15.31K total gates,dissipates 8.6 mW of power and the power delay product is $430e^{-12}$.

Contents

1. Introduction

A crucial part of the orthogonal frequency division multiplexing technology (OFDM) is the Fast Fourier Transform. For FFT processors, the well-known radix-2 technique provides a simple butterfly, but it involves more complex multiplications. In order to simplify twiddle factor multiplication, the radix-2^2, radix-2^3, radix-2^4, and radix-2^k FFT algorithms as well as the higher order radix-4, radix-8, and radix-16 have been developed.

FFT architectures come in two different types: memory-based and pipeline-based. Although the memory-based FFT is small, it requires a larger number of clock cycles to compute the FFT due to the use of a single butterfly. Therefore, their throughput and latency are limited. For radix-2^2 and radix-2^4 algorithms, the pipeline FFT architecture is suggested in [1,2] to minimize the multiplicative complexity. The SDF(Single-path Delay Commutator) is presented in [3,4] memory-less architecture to reduce the area but these are fewer throughputs. The MDF(Multi-path Delay Commutator) architectures [5–8] are efficient regarding memory requirements and throughput but require parallel circuits to compute more samples in parallel and complex control circuitry. A low-power reconfigurable architecture is presented in [9] for

* Corresponding author.
E-mail address: srinivasarao0448@gmail.com (M.S. Rao).

https://doi.org/10.1016/j.vlsi.2024.102244
Received 9 November 2023; Received in revised form 16 July 2024; Accepted 22 July 2024
Available online 24 July 2024

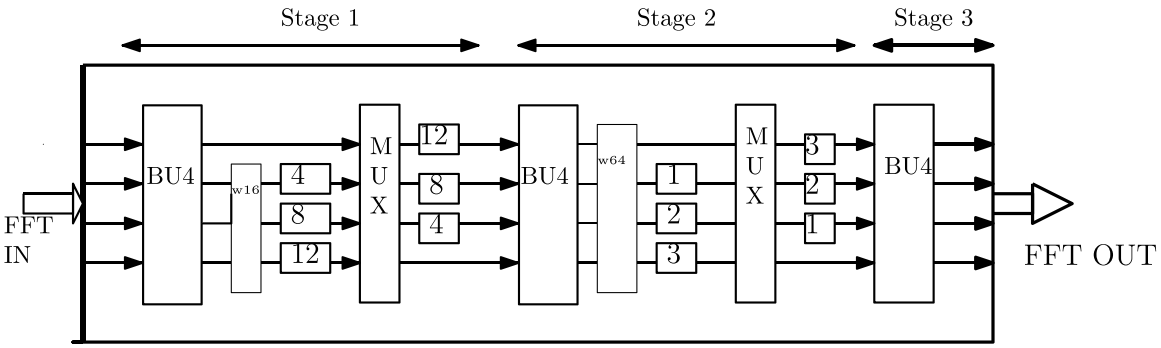

Fig. 1. Proposed 64-point MDC FFT architecture.

IEEE 802.11a-based systems. The multi-path delay commutator(MDC) design was adopted in [10,11] to improve multiplier utilization and throughput. A ROM-less pipelined FFT was suggested by [12] to decrease the hardware required for the design. The authors of [4] have provided a shifter-and adder-based FFT that does not require ROM or multipliers. However the reconfigurable complex multiplier used in the design requires additional space. The MDC design has not been suggested for the memory-less concept to reduce the area. Due to its high throughput and simple to construct control unit, the MDC pipelined FFT architecture is adopted in the proposed work. The twiddle factor must be multiplied with input signals using complex multipliers and the requisite twiddle factors must be stored in ROM, which consumes a lot of space and power. As a result, we developed an FFT with constant multipliers and no ROM to minimize the area.

This paper is organized as follows. The theoretical derivation of algorithm is discussed in Section 2. The Section 3 explains the proposed method of 64-point pipeline FFT Processor. The implementation results and conclusion are discussed in Sections 4 and 5 respectively.

2. Algorithm

The 64-point DFT is

$$X(k) = \sum_{n=0}^{63} x(n) W_{64}^{nk} \tag{1}$$

We can decompose the 64 DFT by Radix-4^2 algorithm

$$n = 16\alpha 1 + 4\alpha 2 + \alpha 3 \qquad \{\alpha 1, \alpha 2, \alpha 3 \in 0, 1, 2, 3\}$$

$$k = \beta 1 + 4\beta 2 + 16\beta 3 \qquad \{\beta 1, \beta 2, \beta 3 \in 0, 1.2, 3\}$$

Then it is rewritten as

$$X(k) = \sum_{\alpha 3=0}^{3} \sum_{\alpha 2=0}^{3} \sum_{\alpha 1=0}^{3} x(16\alpha 1 + 4\alpha 2 + \alpha 3) W_{64}^{(16\alpha 1+4\alpha 2+\alpha 3)(\beta 1+4\beta 2+16\beta 3)}$$

$$= \{W_4^{\alpha 1\beta 1} W_{16}^{\alpha 2(\beta 1+4\beta 2)}\} W_{64}^{\alpha 3(\beta 1+4\beta 2)} W_4^{\alpha 3\beta 3}$$

3. Proposed method

The Radix-4 MDC pipelined FFT for N = 64 is depicted in Fig. 1. The implementation makes use of two complex constant multipliers and the Radix-4 butterfly unit. There is no need for the twiddle coefficient storage area. The features of the proposed MDC architecture are the following:

- o The complex multipliers are replaced by constant multipliers with less area
- o No need of memory to store the twiddle-factors.

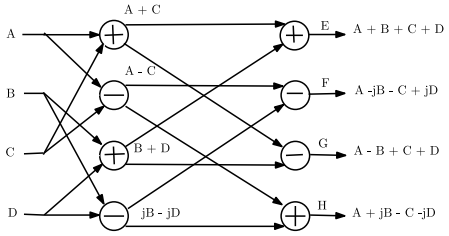

Fig. 2. Radix-4 FFT butterfly.

3.1. Butterfly unit

An equivalent four-point discrete Fourier transform is a radix-4 butterfly unit[BU]. A, B, C, D, and E, F, G, and H are the four complex inputs and outputs, respectively. The structure of the computational element is shown in Fig. 2. There are two stages, each containing eight real adders, a total of 16 adders.

3.2. Complex constant multipliers

The modified complex multiplier in [6] is consider to eliminate the memory to store the twiddle factor storage and to reduce the area of complex multipliers.

3.2.1. W_{16} multiplier unit

For 64-point FFT, it is stated that 8 nontrivial constants W_{16}^{q} are to be multiplied to the intermediate results. As the twiddle factors have 1/8 symmetric property, only 2 sets of (0.9239, 0.3826) and (0.7071, 0.7071) nontrivial constant values are required for the multiplication operation. To compute twiddle factors of three data paths simultaneously, it requires two sets of (0.9239, 0.3826) constants and one set of (0.7071, 0.7071) simultaneously. Each of these constant values is realized by using several shifters and adders. For example, the constant multiplier (0.9239) operation in terms of power of 2 can be decomposed as

$$Y = x \times 0.9239 = x \times \left(1 - 2^{-4} - 2^{-6} + 2^{-9}\right)$$

The procedure of multiplication can be explained as follows. If an input data is to be multiplied with a constant 0.9239. The constant 0.9239 can be decomposed in terms of power of 2 as with an accuracy of 12-bit. Thus, with this representation, the multiplication of x with this constant is done by subtraction of four times right shifted values of x with input x and the result of subtraction again subtraction from the six times shifted values of x and the final output (Y) is obtained from the addition of nine times shifted values of x. The structure of the constant 0.9239 is shown in Fig. 3. This approach is adopted to carry out all the constant multiplication operations.

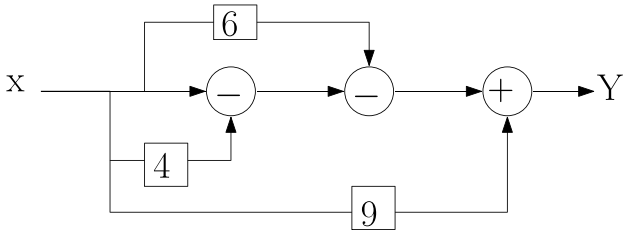

Fig. 3. Structure of constant multiplier.

Table 1
Scheduling of the twiddle factor W_{16}^{q}, where q is from 1 to 2.

Data path/ time slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
2nd	0	0	0	0	1	1	1	1	2	2	2	2	1	1	1	1
3rd	0	0	0	0	2	2	2	2	1	1	1	1	2	2	2	2
4th	0	0	0	0	1	1	1	1	2	2	2	2	1	1	1	1

After exploiting the symmetry property of the twiddle factor, Table 1 presents the scheduling of the twiddle factor in each data path for corresponding time cycle. A total of 16 clock cycles required to compute the 64 point FFT with Radix-4 MDC. At stage 1 and 2, the butterfly outputs need to multiply twiddle factor coefficients. For that purpose, we realized the W_{16}^{q} and W_{64}^{q} complex constant multipliers at 1st and 2nd stage respectively. For the time slot of 0–3 trivial multiplications require pass the input to output. It is evident that Con1 (0.9239, 0.3826) is required simultaneously for the twiddle factors of the 2nd and 4th paths in time slots 4–7 and 12–15. So we need to two sets of Con1 (0.9239, 0.3826) in the time slots from 4–7 and 12–15. In the same way, the second and fourth paths in time slots 8–11 both need for Con2 (0.7071, 0.7071) at the same time. However, as both paths employ the same constant 0.7071, only one con2 sufficient to cover both at once. As a result, our W_{16}^{q} Complex multiplier uses a total of two con1 (0.9239, 0.3826) pairs and one con2 (0.7071, 0.7071) pair (see Fig. 4).

3.2.2. W_{64} multiplier

The W_{64}^{q} multiplication operation can be calculated using just eight sets of nontrivial constant values due to the symmetry property of the twiddle factor. Table 2 shows the scheduling of the twiddle factor in each data path after the twiddle factors are utilizing the symmetry property. It is noticed that the twiddle factor of four paths in time slots 5, 7, 13, and 15 has different values, so one constant pair is sufficient for those time slots. A hardware conflict will occur in time slots 1, 2, 3, 6, 9, 10, and 11, if only one constant multiplier pair is constructed. Therefore, additional constant multiplier pairs are used to prevent hardware conflict. Con2 (0.9808, 0.1951), Con4 (0.9239, 0.3826), Con6 (0.5556, 0.8315), and Con8 (0.7071, 0.7071) are additional pairs that are used in W_{64}^{q}.

Fig. 5 illustrates the block diagram of the improved complex multiplier W_{64}^{q}. The four output sequences from the BU are initially divided into real and imaginary parts. According to the scheduling of the twiddle factor, as depicted in Table 2, the information from each path is sent to the appropriate constant multipliers. With the proper swapping of the real and imaginary parts and the right sign, the full complex multiplication calculation can be performed using just fourteen sets of constant values. Comparing this approach to the four-complex-multiplier approach, the gate count can be reduced by around 30% while maintaining performance as per with the four complex multipliers.

Table 2
Scheduling of the twiddle factor W_{64}^{q}, where q is from 1 to 8.

Data path/ time slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1st	0	0	0	0	0	1	2	3	0	2	4	6	0	3	6	7
2nd	0	4	8	4	0	5	6	1	0	6	4	2	0	7	2	5
3rd	0	8	0	8	0	7	2	5	0	6	4	2	0	5	6	1
4th	0	4	8	4	0	3	6	7	0	2	4	6	0	1	2	3

4. Comparison and experimental results

A 64-point FFT computation hardware requirements and performance for the proposed MDC designs are compared to those of comparable pipelined architectures in Table 3. The multipliers are adopted by the modified constant multiplier in [6], therefore the proposed architectures do not need any complex multipliers. The proposed architecture does not require memory to store the twiddle factors that reduce the total area of the FFT design.

Even though the Radix-2^k architectures in [3] have fewer adders, the memory demand is higher, as seen in Table 3. However, although having the same amount of internal memory as the suggested architecture, the architecture in [13] needs RAM to hold the twiddle factor coefficients. The proposed design has a throughput that is 4 times higher and a lower latency than the architecture in [3].

The Proposed FFT Architecture is modelled in VHDL and the functional verification is carried out in Matlab environment. After the functional validation, the proposed architecture was synthesized in cadence Genus software with TSMC 45 nm,1.1 V Standard Cell Library. Table 4 compares the hardware of the proposed architecture with other architectures. In comparison to the designs presented in [3,4,12–14], the proposed R4^2 MDC architecture has fewer gates since it uses a modified constant multiplier. For meaningful comparison, we calculated the normalized power and power delay product. The normalized power per FFT is considered as follows [3]:

$$\text{Normalized power per FFT} = \frac{Power}{(Voltage)2 * FFTsize * Frequency} * 1000$$

The normalized power of the proposed design is more compared to designs presented in [3,4,8] whereas less compared to designs presented in [9,12,14]. The proposed design consumes 8.6 mW of power for four spatial streams at a clock frequency of 20 MHz and has 15 311 gates.

5. Conclusion

In this paper, we present Radix-4^2 MDC pipelined 64-point FFT architectures that are suitable for OFDM based wireless systems. The proposed architecture employ radix-4^2 algorithm and pipelined MDC architecture that provide lower multiplicative complexity and less hardware. The hardware cost of the architecture is further reduced as the complex multiplier is realized by using modified constant multipliers. The synthesized results show that at 20 MHz frequency the proposed Radix-4^2 MDC architecture occupies 15.31 K total gates and dissipate only 8.6 mW of power. It can operate at maximum frequency of 83 MHz with total gate count of 17.378 K and lower power delay product of $430e^{-12}$.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

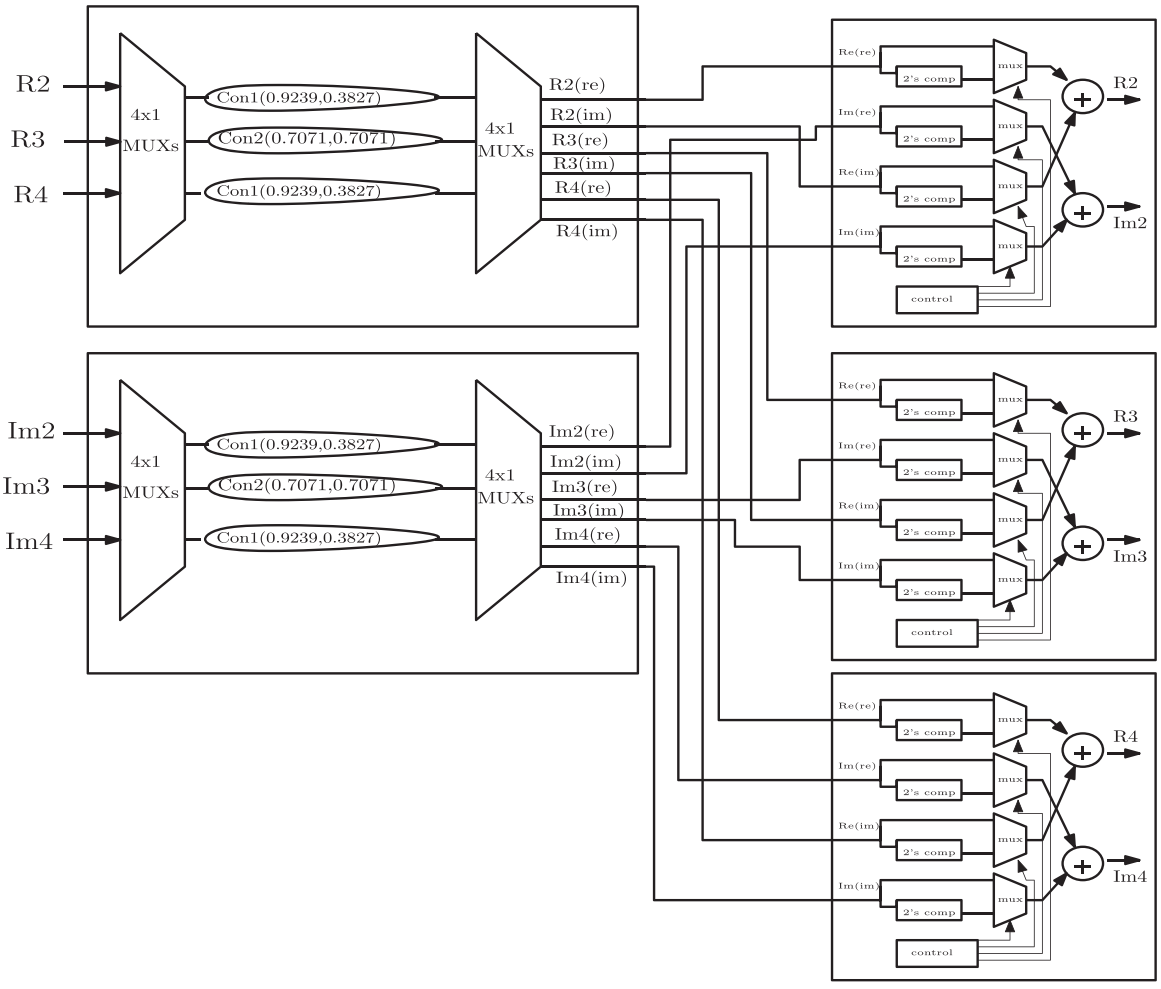

Fig. 4. Block diagram of the W_{16}^{q} Complex multiplier.

Table 3
Comparison of the proposed Radix-4^2 MDC architecture to other architectures for the computation of a 64-point FFT.

Architecture	Complex multipliers	Constant multipliers	Adders	Memory	Latency	Throughput
Radix-4 MDC [13]	3	0	24	60	15	4R
Radix-2^2 SDF [3]	0	2	12	63	127	R
Radix-2^3 SDF [3]	0	3	12	63	127	R
Radix-2^4 SDF [3]	0	2	12	63	127	R
Proposed Radix-4^2 MDC	0	2	24	60	15	4R

Table 4
Comparison of the 64-point pipelined FFT architecture.

Design	Word length	Gate count	Technology	Power	Normalized power	P.D product
[3]	16	28 880	180 nm, 1.8 V	7.86 mw @ 20 MHz	1.89	$393e^{-12}$
[4]	16	33 590	180 nm, 1.8 V	9.79 mw @ 20 MHz	2.36	$489.5e^{-12}$
[12]	16	–	250 nm, 1.8 V	41 mw @ 20 MHz	9.88	$2050e^{-12}$
[14]	In12, Out20	38 168	350 nm, 3.3 V	507 mw @ 150 MHz	4.84	$3346e^{-12}$
[13]	–	39 506	0.35um	–	–	–
[8]	In8, out24	3086k	90 nm	682 mw @ 450 MHz	$2.9e^{-5}$	–
[9]	16	–	180 nm	58.27 mw @ 20 MHz	4.55	$227.5e^{-12}$
Proposed	In12, Out16	15 311	45 nm, 1.1 V	8.6 mw @ 20 MHz	5.55	$430e^{-12}$
Proposed	In12, Out16	17 378	45 nm, 1.1 V	22.37 mw @ 83.3 MHz	3.46	$268.44e^{-12}$

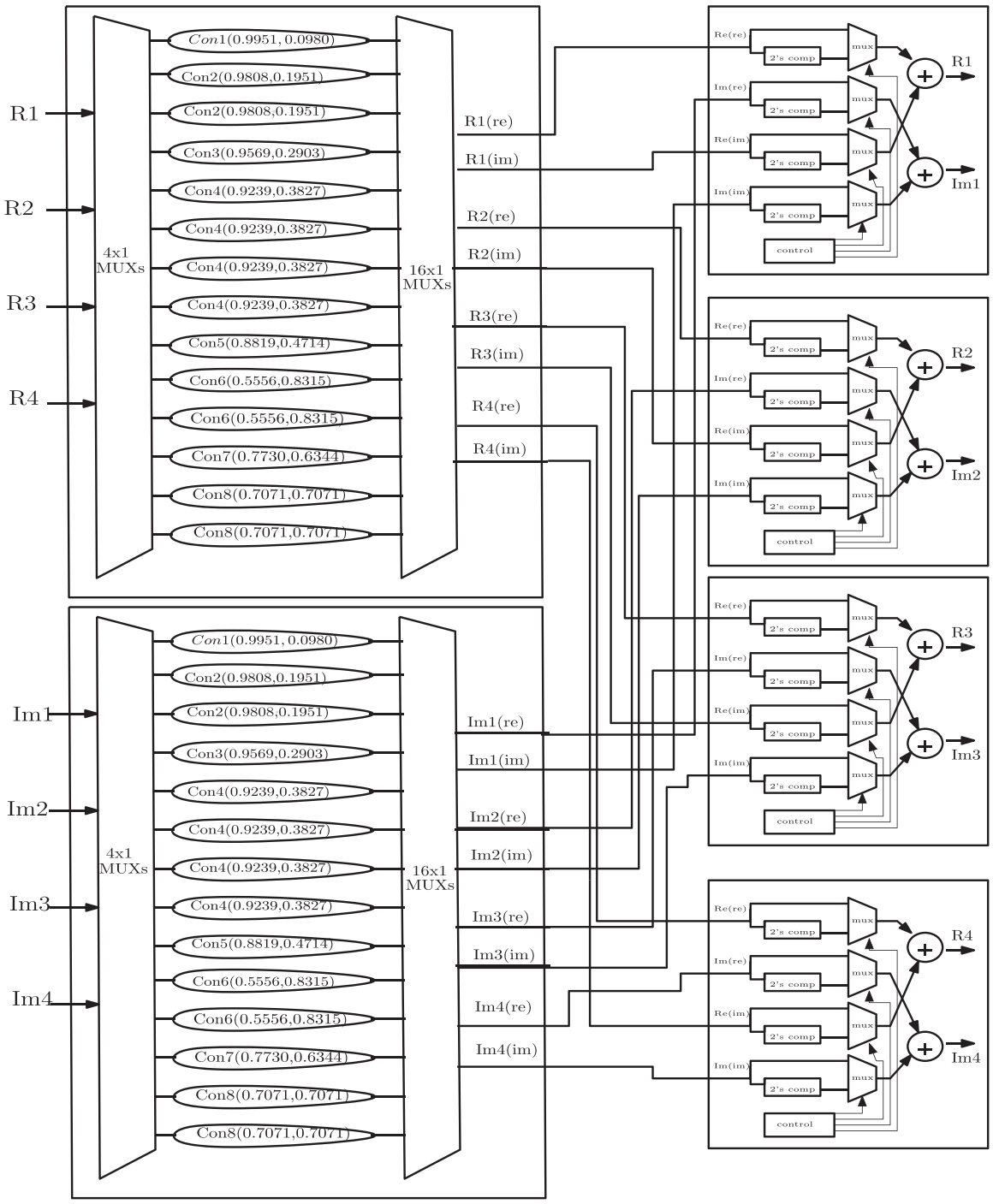

Fig. 5. Block diagram of the W_{64}^{q} Complex multiplier.

References

[1] Shousheng He, Mats Torkelson, A new approach to pipeline FFT processor, in: Proceedings of International Conference on Parallel Processing, IEEE, 1996, pp. 766–770.

[2] Jung-Yeol Oh, Myoung-Seob Lim, New radix-2 to the 4th power pipeline FFT processor, IEICE Trans. Electron. 88 (8) (2005) 1740–1746.

[3] Ganesh Kumar Ganjikunta, Subhendu Kumar Sahoo, An area-efficient and low-power 64-point pipeline fast Fourier transform for OFDM applications, Integration 57 (2017) 125–131.

[4] Chu Yu, Mao-Hsu Yen, Pao-Ann Hsiung, Sao-Jie Chen, A low-power 64-point pipeline FFT/IFFT processor for OFDM applications, IEEE Trans. Consum. Electron. 57 (1) (2011) 40.

[5] Hang Liu, Hanho Lee, A high performance four-parallel 128/64-point radix-2^4 FFT/IFFT processor for MIMO-OFDM systems, in: APCCAS 2008-2008 IEEE Asia Pacific Conference on Circuits and Systems, IEEE, 2008, pp. 834–837.

[6] Yu-Wei Lin, Hsuan-Yu Liu, Chen-Yi Lee, A 1-gs/s fft/ifft processor for uwb applications, IEEE J. Solid-State Circuits 40 (8) (2005) 1726–1735.

[7] Yu-Wei Lin, Chen-Yi Lee, Design of an FFT/IFFT processor for MIMO OFDM systems, IEEE Trans. Circuits Syst. I. Regul. Pap. 54 (4) (2007) 807–815.

[8] Hong-Ke Lin, Pin-Han Lin, Chih-Wei Liu, Design of a high-throughput and area-efficient ultra-long FFT processor, in: 2020 International Symposium on VLSI Design, Automation and Test, VLSI-DAT, IEEE, 2020, pp. 1–4.

[9] V. Sarada, E. Chitra, Low-power reconfigurable FFT/IFFT processor, in: Intelligent Data Communication Technologies and Internet of Things: Proceedings of ICICI 2020, Springer, 2021, pp. 291–300.

[10] Mario Garrido, Jesús Grajal, MA Sanchez, Oscar Gustafsson, Pipelined radix-2^k feedforward FFT architectures, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 21 (1) (2011) 23–32.

[11] Kai-Jiun Yang, Shang-Ho Tsai, Gene C.H. Chuang, MDC FFT/IFFT processor with variable length for MIMO-OFDM systems, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 21 (4) (2012) 720–731.

[12] Koushik Maharatna, Eckhard Grass, Ulrich Jagdhold, A 64-point Fourier transform chip for high-speed wireless LAN application using OFDM, IEEE J. Solid-State Circuits 39 (3) (2004) 484–493.

[13] Yunho Jung, Hongil Yoon, Jaeseok Kim, New efficient FFT algorithm and pipeline implementation results for OFDM/DMT applications, IEEE Trans. Consum. Electron. 49 (1) (2003) 14–20.

[14] Wen-Chang Yeh, Chein-Wei Jen, High-speed and low-power split-radix FFT, IEEE Trans. Signal Process. 51 (3) (2003) 864–874.